

# Chapter 05. フォームの基礎知識

## I. 提出物について

### I.1 基礎課題

- 以下の内容を満たす demo02 フォルダを作成し, studydata の自身のフォルダにアップロードすること

#### [1] html ファイル

- jQuery ライブラリと js ファイルを読み込むコードを記述

#### [2] js ファイル

- 以下のコードを記述
  - 例題 1 (1) 【8 ページ】, 練習 1 (1) 【8 ページ】
  - 例題 2 (1) 【9 ページ】, 練習 2 (1) 【9 ページ】
  - 例題 3 (1) 【9 ページ】, 練習 3 (1) 【13 ページ】
  - 例題 4 (1), (2) 【15 ページ】
  - 例題 5 (1), (2) 【20 ページ】

## 1.2 実習環境の構築（jQuery04 を編集）

Step 1. demo03 を jQuery04 のフォルダにコピー

Step 2. main.js のコードを以下の部分だけ残して保存

```
1  // 以下のコードはいじらないでください
2  // access.htmlのheaderのデザインを修正
3  $(function() {
4      function applyStyle() {
5          if ($(window).width() >= 800) {
6              // 800px以上の時に適用するスタイル
7              $('.site-menu ul li').css({
8                  'height': '110px',
9                  'margin': '0',
10                 'padding': '0 20px',
11                 'display': 'flex',
12                 'alignItems': 'center'
13             });
14         } else {
15             // 800px未満になったら元に戻す
16             $('.site-menu ul li').css({
17                 'height': '',
18                 'margin': '',
19                 'padding': '',
20                 'display': '',
21                 'alignItems': ''
22             });
23         }
24     }
25
26     applyStyle(); // 初回ロード時に実行
27     $(window).on('resize', applyStyle); // 画面サイズ変更でも実行
28 });
```

## I. フォームの基礎知識

- サーバーにデータを送信する方法としてフォームが存在する
- メールアドレスや電話番号など入力した内容などが送信可能
- フォームでは主に `get`, `post` と呼ばれるメソッドを設定して運用する

### I.1 get メソッド

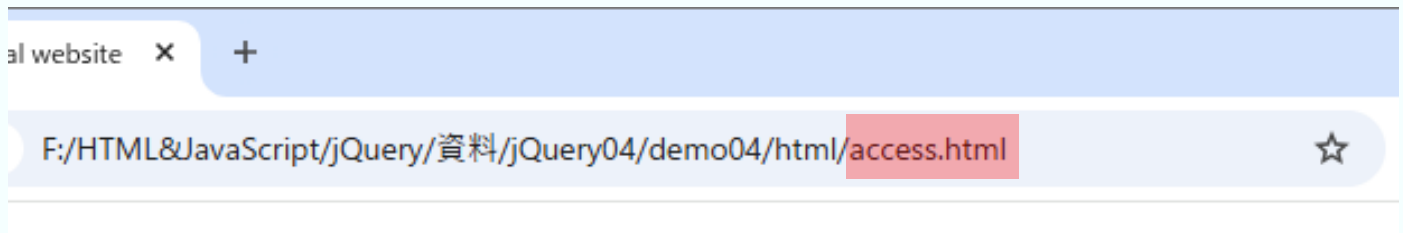
- 主に動作確認やデバッグ目的で使用
- フォームで入力した内容を URL の後ろに付与してサーバーに送信
- URL に入力内容が表示されるため、履歴やログから入力した内容が漏れやすいため、個人情報や重要なデータを送信する用途で `get` メソッドを使用してはいけない

※ ブラウザで検索をかけるときにも用いられる

#### 【基本構文】

```
1 <h2>お問い合わせフォーム</h2>
2 <form action="#" method="get">
```

## 【サーバーにデータを送る前の URL】



## 【get メソッドでサーバーにデータを送った後の URL】

お問い合わせフォーム

---

お名前 必須

鎌田

メールアドレス 必須

kamada@ncs.com

お電話番号

123456789

お問い合わせ種別

ご予約について ▼

お客様について

☒ 一般のお客様  
☐ お取引先様  
☐ その他

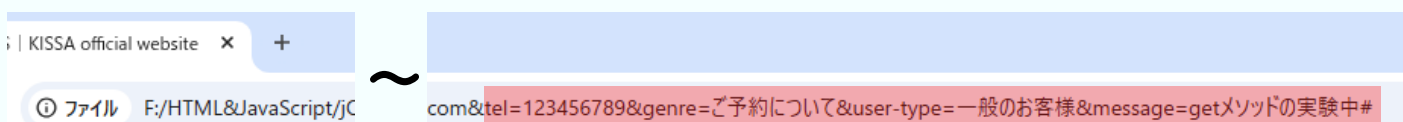
お問い合わせ内容 必須

getメソッドの実験中

ご入力内容をご確認の上、お間違いがなければ【Submit】ボタンを押してください。

Submit

- 上記の内容で送信すると…



- URL の後ろに input で入力した情報が付与される！

## 1.2 post メソッド

- 主に本番の実装で使用
- フォームで入力した内容を HTTP 通信の body 部分に含める形でサーバーに送信
- get メソッドと違って URL に入力した内容が付与されないので通信内容を直接確認できない
- ただし、完全に秘匿できるわけではないので、HTTPS 通信など別の手法を併用して実装する

### 【基本構文】

```
1 <h2>お問い合わせフォーム</h2>
2 <form action="#" method="post">
```

### 【post メソッドの利用例】

お問い合わせフォーム

お名前 必須

メールアドレス 必須

お電話番号

お問い合わせ種別

お客様について ☒ 一般のお客様  
☐ お取引先様  
☐ その他

お問い合わせ内容 必須

ご入力内容をご確認の上、お間違いがなければ【Submit】ボタンを押してください。

Submit

127.0.0.1:5500/html/access.html#



このページは動作していません

この問題が何度も発生する場合は、サイト所有者にお問い合わせください。

HTTP ERROR 405

- 送信先のサーバを指定していないのでエラーが出る
  - post を実際に動かすためには、action の値に適切なサーバーへのパスを記述する必要がある
- ※ GET も POST もサーバーへ送信するが、POST はサーバー側が未対応だとエラーになりやすい

## 2. イベントオブジェクト (e)

- on メソッドや addEventListener (JavaScript のメソッド) を用いてクリックなどの操作を行うと、イベントオブジェクトが自動的に生成される
- イベントオブジェクトには、以下のような情報が格納されている。
  - イベントの種類 (クリック, スクロールなど)
  - イベントが発生した要素
  - マウスカーソルの座標

### 【基本構文】

```
1  $('セレクト名').メソッド名(function(e){  
2  
3  });
```

### 【例: on メソッドと併用したパターン】

```
1  $(function(){  
2      $('input').on('click', function(e){  
3          console.log(e);  
4      });  
5  });
```

## 【console.log で取得できる情報】



## 【例：イベントオブジェクトを利用して、クリックした個所の座標を取得】

- 上記の画像より、x 軸の座標を表しているプロパティは screenX, y 軸は screenY である
- この情報を取得するには、e.screenX, e.screenY という形式で記述すればよい

```
1  $(function(){
2      $('input').on('click', function(e){
3          alert(
4              'X座標の値は: ' + e.screenX +
5              '\nY座標の値は: ' + e.screenY
6          );
7      });
8  });
```

## 【実行結果】





## 例題1. イベントオブジェクトを利用して様々なデータを確認してみよう

- (1) inputタグの type 属性が text の要素をクリックしたときに発生するイベントオブジェクト e の内容を console.log で取得しなさい

```
1  $(function(){
2      $('input[type=text]').on('click', function(e){
3          console.log(e);
4      })
5  });
```

## 練習1. 以下のコードを実装しなさい

- (1) textarea 要素を blur したときに発生するイベントオブジェクト e を console.log で取得しなさい

### 3. code (キー入力の判定)

- JavaScript や jQuery では、キーボードの各キーに対応した code と呼ばれる値が定められている
- 特定のキーを押したときに、何らかの処理を実行することができる。
- if 文や on メソッドと組み合わせて使用されることが多い

#### 【対応表】

| キーボードのキー | jQuery で動かすための値 |
|----------|-----------------|
| 1        | Digit1          |
| 2        | Digit2          |
| a        | KeyA            |
| b        | KeyB            |
| enter    | Enter           |
| space    | Space           |

※ 以前は keyCode が使用されていたが、現在は 非推奨 (deprecated) であるため、本授業では使用しない。

※ 昨年度まで keyCode を使用しているが、今年度は code を用いた記述に統一する。

昨年度の資料 > jQuery を利用したフォームの作成

#### 例題2. イベントオブジェクトと code を利用したコードを実装してみよう

(1) input 要素を選択している状態で enter を押したときに、alert で文字を表示するコード

```
1  $(function () {  
2    // keydownは、要素が選択されているときにキーが押されたときに実行される  
3    $('input').on('keydown', function (e) {  
4      // e.codeで入力されたキーを取得  
5      if (e.code === 'Enter') {  
6        alert('enterが押されました');  
7      }  
8    });  
9  });
```

#### 練習2. 以下のコードを実装しなさい

(1) input 要素を選択している状態で a を入力したときに、alert で 'a が押されました' と表示するコード

## 4. find メソッド

- `$('セレクト名')`で選択した要素を起点として、その内部（子孫要素）から指定した要素を取得するメソッド
- `$('.box').find('p')`は、`.box` の中にある `p` 要素だけをまとめて取得できる
- 上記の文であれば、`$('.box p')`のようにセレクトでも同じ指定はできるが、  
`find()`は起点（`.box` など）を引数で差し替えやすいため、  
関数化して再利用する際に範囲を柔軟に変更できる
- さらに、前段の選択結果を引き継げるため、  
`$('#top').addClass('active').find('p').hide()`のように  
「親の処理→子の処理」を続けて記述できる

### 【基本構文】

```
1  $('セレクト名').find('探したい要素名');
```

- 後述する `each` メソッドと併用すること多い

【例: `class:main` の中にある, `type` 属性が `text` の個所の `input` タグだけを取得】

```
1  $('.main').find('input[type=text]');
```

## 5. each メソッド

- 取得した複数の要素を 1 つずつ順番に処理するためのメソッド
- 4 つの li 要素がある場合、1 番目の li から順番に何らかの処理を実行することが可能
- for 文や while 文でも同じ処理を記述することができるが、  
要素数を数えたり繰り返し処理用の変数を用意したりしなければいけないので処理が冗長になる

### 【基本構文】

```
1  $('セレクト名').each('処理したい内容');
```

### 【例: 4 つの li タグに対して、順番に何らかの処理を実行】

▶ each メソッドを使って記述する場合

```
1  $('li').each(function(){  
2      // 実行したい処理を記述  
3  });
```

▶ for 文を使って記述する場合

```
1  const $li = $('li'); // li要素をまとめて変数$liに格納  
2  // $li.lengthで$liに格納されている要素の数を取得  
3  for (let i=0; i<$li.length; i++){  
4      // 実行したい処理を記述  
5  }
```

- 手間が 2 つくらい増えるため、each メソッドを使って簡潔に表現することが多い

## 【例:find メソッドと併用したケース】

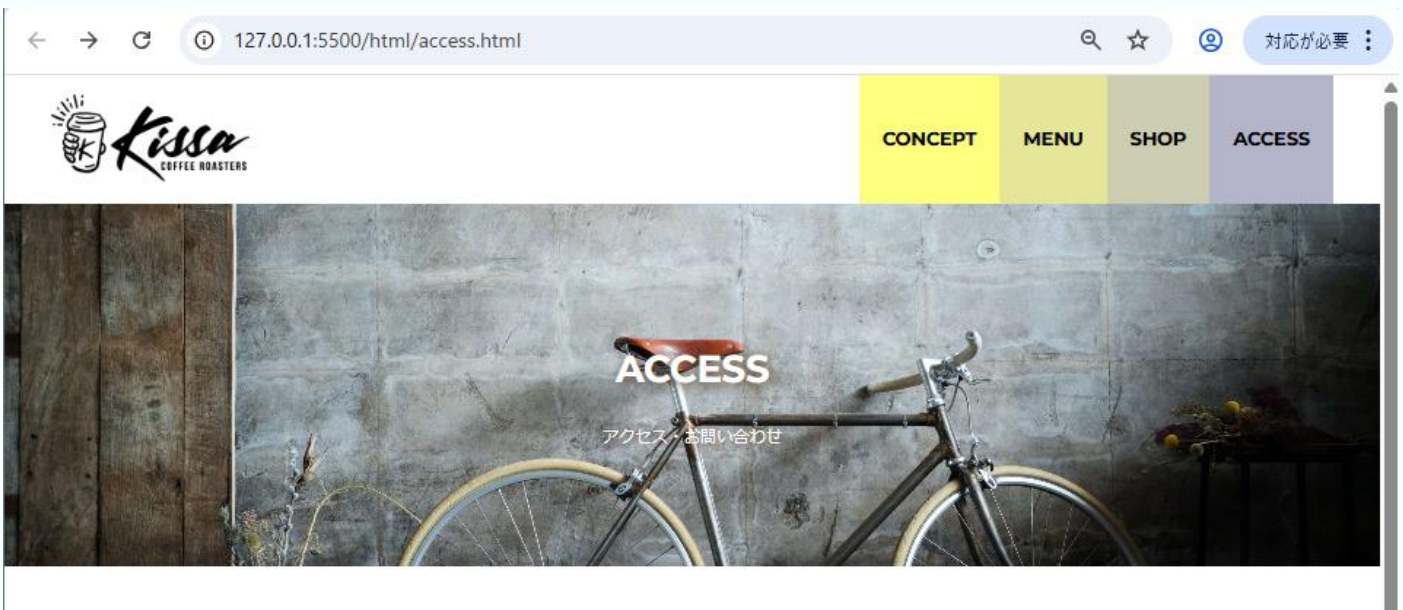
```
1  $('main').find('input[type=text]').each(  
2      function(){  
3          let size = 10;  
4          $(this).css({fontSize: size})  
5          size = size + 10;  
6      }  
7  );
```

- メソッドチェーンを用いる、このような記法をよく用いる
- \$('セレクト名')の箇所をいじることで、配列やオブジェクトを操作することも可能
- ただ、配列・オブジェクトを操作する場合、\$.each(引数, 処理)という書き方を用いることを推奨
  - \$('セレクト名').each(); ← jQuery オブジェクトとして操作
  - \$.each(引数, 処理); ← 配列やオブジェクトとして操作

### 例題3. find メソッドと each メソッドを使ってみよう

- (1) class:header-site-menu 内の li 要素を each メソッドで 1 つずつ取得し、背景色がだんだんと変化するコードを記述しなさい

```
1 $(function(){
2   // 背景色の値を設定
3   let col_1 = 255;
4   let col_2 = 255;
5   let col_3 = 0;
6
7   $('.header-site-menu').find('li').each(
8     // eachメソッドで、選択された各要素に対して以下の処理を実行
9     function(){
10      $(this).css({
11        backgroundColor: 'rgba(' + col_1 + ', ' + col_2 + ', ' + col_3 + ', 0.5)'
12      });
13
14      // 背景色を切り替えた後、rgbaに設定する値を計算
15      col_1 = col_1 - 50;
16      col_2 = col_2 - 50;
17      col_3 = col_3 + 50;
18    });
19  });
```



### 練習3. 以下のコードを実装しなさい

- (1) 以下の内容 class:footer 内の li 要素を each メソッドで 1 つずつ取得し、文字色がだんだんと変化するコードを記述しなさい

## 6. prop メソッド

- 指定した要素の属性やプロパティを取得・設定するためのメソッド

| 属性    | プロパティ    |
|-------|----------|
| class | selected |
| id    | checked  |
| name  | required |

※ 属性をメインに扱う attr メソッドも存在する

### 【例: 指定した属性の値を取得するコード】

```
1  $('セレクト名').prop('属性');  
2  
3  // 利用例  
4  $('nav').prop('class');
```

### 【例: 指定したプロパティがあるか確認するコード】

```
1  $('セレクト名').prop('プロパティ');  
2  
3  // 利用例  
4  $('nav').prop('required');
```

- プロパティを指定した場合、それが存在するか否かという true / false を返す
- 第2引数に true / false の真偽値を入れることで、プロパティの設定をすることが可能

### 【例: 属性を設定するコード】

```
1  $('セレクト名').prop('属性', 設定したい値・関数);  
2  
3  // 利用例  
4  $('footer').prop('id', 'test');
```

※ prop の第 2 引数には関数を入れることもできる

## 7. val メソッド

- フォーム部品 (input, textarea, select など) に  
入力された値 (value) を取得・設定するためのメソッド
- 入力チェックや送信前の確認など, フォーム処理の中心になる。

### 例題4. prop メソッドと val メソッドを使ってみよう

(1) prop メソッドを使って、header 要素に id:test を付与するコードを記述しなさい

```
1  $(function(){
2      $('header').prop('id', 'test');
3  });
```

(2) val メソッドを使って blur したときに、入力した内容を console.log で出力するコードを記述しなさい

```
1  $(function(){
2      $('input').on('blur', function(){
3          console.log($(this).val());
4      });
5  });
```



## 8. before メソッド

- 指定した要素の直前に新しい要素や文字列を挿入するメソッド
- ※ 指定した要素の直後に要素を追加する after メソッドも存在する

### 【基本構文】

```
1 $('セレクト名').before('追加したい内容');
```

【例:span 要素に対して、before メソッドを使って文字を追加】

```
1 $('span').before('これはspan要素です');
```

これはspan要素ですお名前 必須

これはspan要素ですメールアドレス 必須

## 【例:h2 要素の後ろに、p タグを使って文字を追加】

```
1 $('h2').before('<p>h2タグの後ろに文章が記述されています</p>')
```

h2タグの後ろに文章が記述されています

### アクセスマップ

---



h2タグの後ろに文章が記述されています

### お問い合わせフォーム

---

## 9. remove メソッド

- 指定した要素を削除するメソッド
- 削除後は要素自体が無くなるため、  
元に戻したい場合は表示・非表示 (show / hide) を使う方法を検討する

### 【基本構文】

```
1  $('セレクト名').remove();
```

### 【すべての h2 タグを削除】

```
1  $('h2').remove();
```

## 10. addClass メソッド

- 指定した要素に class を追加するメソッド
- prop メソッドでも同様の処理が可能だが、  
クラス名を修正する場合、意図が明確なこちらのメソッドを利用することが望ましい
- cssと組み合わせることで、見た目の切り替えを簡単に実装できる

### 【基本構文】

```
1 $('セレクト名').addClass('追加したいclass名');
```

ご入力内容をご確認の上、お間違いがなければ  
[Submit] ボタンを押してください。

Submit

CONCEPT

```
<!DOCTYPE html>
<html> scroll
  <head> ... </head>
  <body>
    <!-- headerここから -->
    <header class="header"> ... </header>
    <!-- headerここまで -->
    <!-- mainここから -->
    <main class="main"> ... </main>
    <!-- mainここまで -->
    <!-- footerここから -->
    ... <footer class="footer test"> ... </footer>
    <!-- footerここまで -->
    <!-- Code injected by live-server -->
    <script> ... </script>
  </body>
</html>
```

### 【test という class 名を, footer に追加するコード】

```
1 $('footer').addClass('test');
```

## 11. removeClass メソッド

- removeClass メソッドは, 指定した要素から 特定の class を削除するメソッドである。
- 使い方は addClass メソッドと同じ

```
1 .find('取得したい要素名')
```

- 教科書で作成した web ページでは、これらを活用して  
ナビゲーションメニューが出たり消えたりする仕組みを実装している

例題5. access.css に以下のコードを追加した後に、クラスの付与と削除を試してみよう

事前準備: access.css に次のコードを記述

```
1  .click {
2    color: red;
3  }
```

(1) h2 要素をクリックしたときに、クリックした要素に click という class 名を付与

```
1  $(function(){
2    $('h2').on('click', function(){
3      $(this).addClass('click');
4    });
5  });
```

(2) (1) で作成したコードを改変して、再度 h2 要素をクリックしたときに、  
(1)で付与した click という class 名を削除

```
1  $(function(){
2    $('h2').on('click', function(){
3      // h2にclickというクラスを持っているとき
4      if($(this).hasClass('click')){
5        // clickというクラスを削除
6        $(this).removeClass('click');
7      } else {
8        // clickというクラスを付与
9        $(this).addClass('click');
10     }
11   });
12 });
```